

作业：前端开发

分工说明

本次作业按角色分工完成，负责前端的同学只需完成前端部分。部署（Docker）由后端同学负责，前端无需编写 Dockerfile。

作业目标

完成项目前端的核心页面开发，实现与设计稿一致的界面，并与后端 API 完成对接。

具体要求

以下要求适用于所有技术栈（Web / 小程序 / Flutter / Android / iOS），请按实际使用的平台理解对应内容。

1. 核心页面开发

完成项目所有核心页面：

- 登录/注册页面
- 首页/列表页面
- 详情页面
- 个人中心
- 其他业务页面（根据项目需求）

2. 组件/模块封装

封装可复用的 UI 组件或功能模块（至少 3 个），保持合理的目录结构：

技术栈	组件/模块示例
React / Vue	通用组件（Button、Modal、列表卡片）
小程序	自定义组件（navigation、card、form）
Flutter	Widget 封装（AppBar、ListItem、Dialog）
Android	自定义 View / Fragment / ViewModel

3. 后端 API 对接

- 封装统一的网络请求层（HTTP 客户端 / SDK）
- 先用 Mock 数据开发，待后端 API 就绪后替换
- 处理加载状态、错误提示、鉴权跳转

技术栈	推荐方式
React / Vue	axios / fetch 封装
小程序	wx.request 封装
Flutter	http / dio 包
Android	Retrofit / OkHttp

💡 与后端同学约定好接口路径和数据格式后再对接，减少返工。

4. 代码规范

- 统一代码风格（格式化工具或团队约定）
- 关键逻辑添加注释

目录结构

根据技术栈不同，前端项目放在 `frontend/` 目录下，保持结构清晰即可：

```
project/
├── frontend/                                # 前端项目（本次作业范围）
│   └── ...（按技术栈组织）
├── docs/
│   ├── design/                            # 设计稿截图
│   └── contributions/
│       └── 05-frontend/
│           ├── zhangsan.md
│           └── lisi.md
└── README.md
```

提交内容

Git 仓库提交

1. 前端项目完整代码（`frontend/` 目录）
2. 页面截图保存到 `docs/design/screenshots/`
3. `docs/contributions/05-frontend/你的名字.md`

学习通提交

提交以下截图作为个人完成证明：

1. Git 提交记录 — `git log --author="YOUR_NAME" --oneline` 终端截图
2. 页面运行截图 — 项目成功运行 + 至少 5 个页面/界面截图
3. API 对接截图 — 网络请求成功返回后端数据的截图（若后端已就绪）

提交截止

下次课前

个人贡献说明

每位前端成员在 `docs/contributions/05-frontend/` 下提交贡献说明：

文件名：你的名字.md

内容模板：

```
# 前端开发贡献说明

姓名：XXX
学号：XXX
技术栈：XXX（如 React / 小程序 / Flutter / Android）
日期：XXXX-XX-XX

## 我完成的工作

### 页面开发
- [ ] 登录/注册页面
- [ ] 首页/列表页面
- [ ] 详情页面
- [ ] 个人中心
- [ ] 其他：XXX

### 组件/模块封装
- 组件 1：XXX（说明用途）
- 组件 2：XXX
- 组件 3：XXX

### API 对接
- [ ] 封装网络请求层
- [ ] 对接后端接口（接口名称：XXX）
- [ ] 处理加载状态和错误

## PR 链接

- PR #X: https://github.com/xxx/xxx/pull/X

## 遇到的问题和解决

1. 问题：XXX
   解决：XXX
```

心得体会

(简要说明前端开发过程中的收获)

评分标准

项目	分值	说明
页面完成度	35%	完成所有核心页面，与设计稿一致
组件封装	20%	至少 3 个可复用组件，结构合理
API 对接	20%	请求封装完整，有加载/错误处理
代码规范	10%	代码风格统一，关键逻辑有注释
个人贡献说明	15%	贡献描述清晰，有截图证明

额外加分项

- 良好的用户体验 (加载动画、空状态、错误提示)
- 响应式/自适应布局 (不同屏幕尺寸适配)
- 页面过渡动画
- 统一状态管理 (Pinia / Redux / Provider / ViewModel)
- 使用类型系统 (TypeScript / Dart 强类型 / Kotlin)